

Trabajo Práctico 2 — Gwent

[75.07/95.02/TB025] Paradigmas de Programación

Curso 01

Primer cuatrimestre de 2025

Grupo 09

Corrector: Suárez Pablo

Alumno:	Número de padrón:	Email:
Agustin Ivan Abitabile	111081	aabitabile@fi.uba.ar
Axel Miguel Billordo	107167	abillordo@fi.uba.ar
Felipe Ignacio Quero R.	111374	fquero@fi.uba.ar
Santiago Fardini	108692	sfardini@fi.uba.ar
Tomás Bautista Conti	111760	tconti@fi.uba.ar

Índice

1. Introducción	2
Objetivo	2
Consigna General	2
Motivación/Background	2
2. Supuestos	2
3. Diagramas de clases	2
4 Detalles de implementación	3
5. Diagramas de secuencia	3
6. Diagrama de paquetes	3
7. Excepciones	3

1. Introducción

El presente informe reúne la documentación de la solución del segundo trabajo práctico de la materia Algoritmos y Programación III que consiste en desarrollar el juego Gwent que se trata de un juego de cartas de dos jugadores que consiste en 3 rondas, donde el jugador que gane dos rondas gana el juego.

2. Supuestos

- Las secciones no tienen límite de cartas.

3. Diagramas de clases

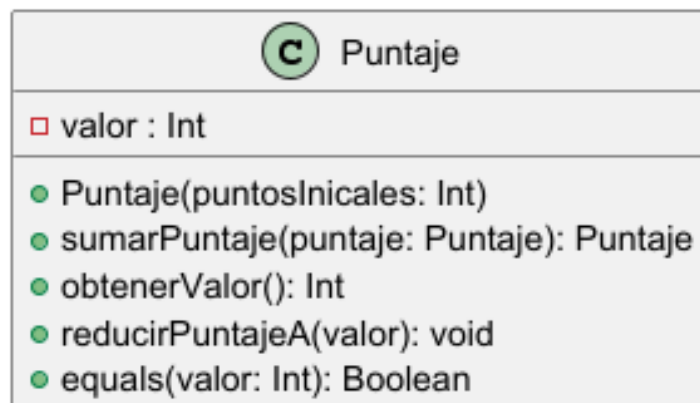


Figura 1: Diagrama de clases de Puntaje

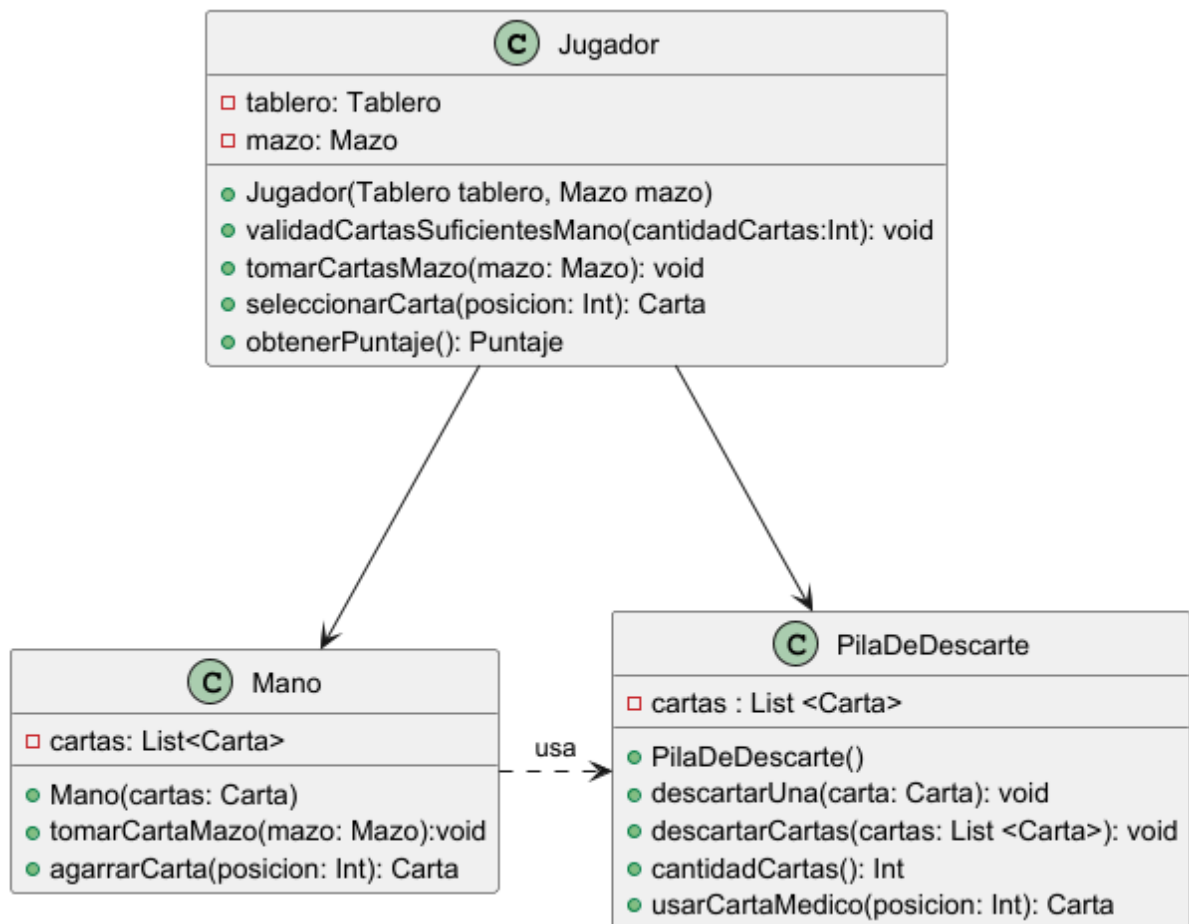


Figura 2: Diagrama de clases de Jugador

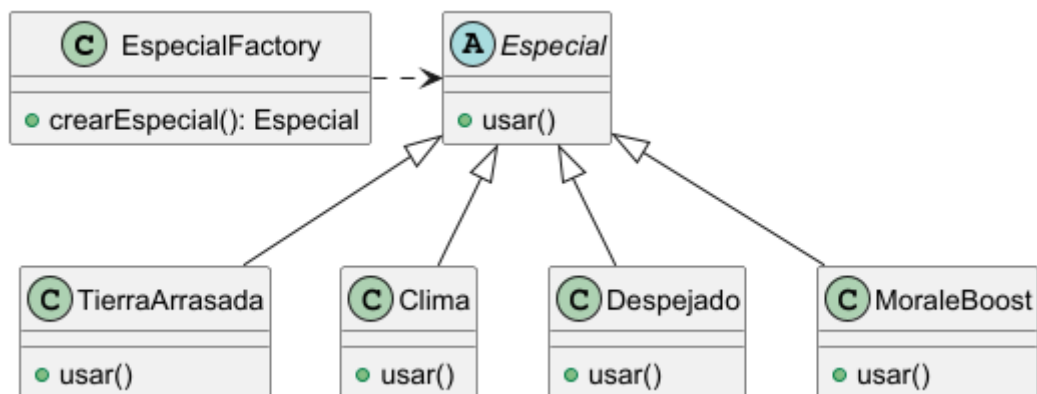


Figura 3: Patrón Factory para las cartas Especiales

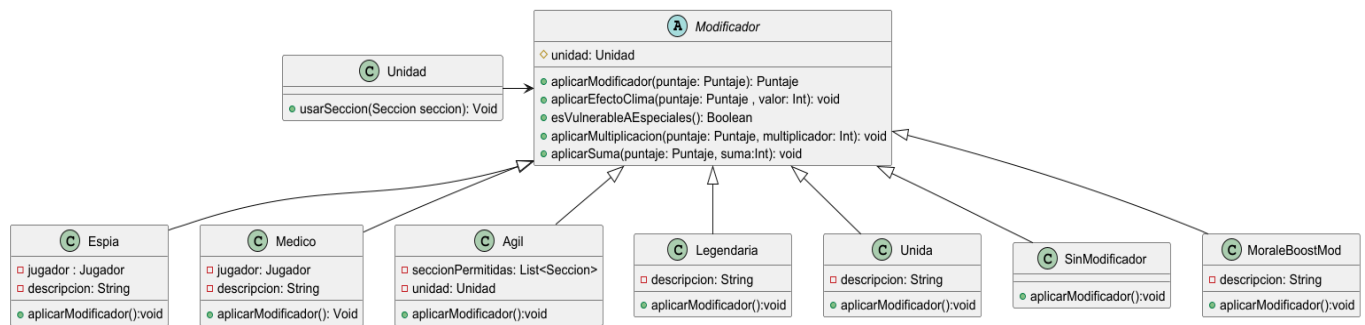


Figura 4: Patrón Strategy para los Modificadores

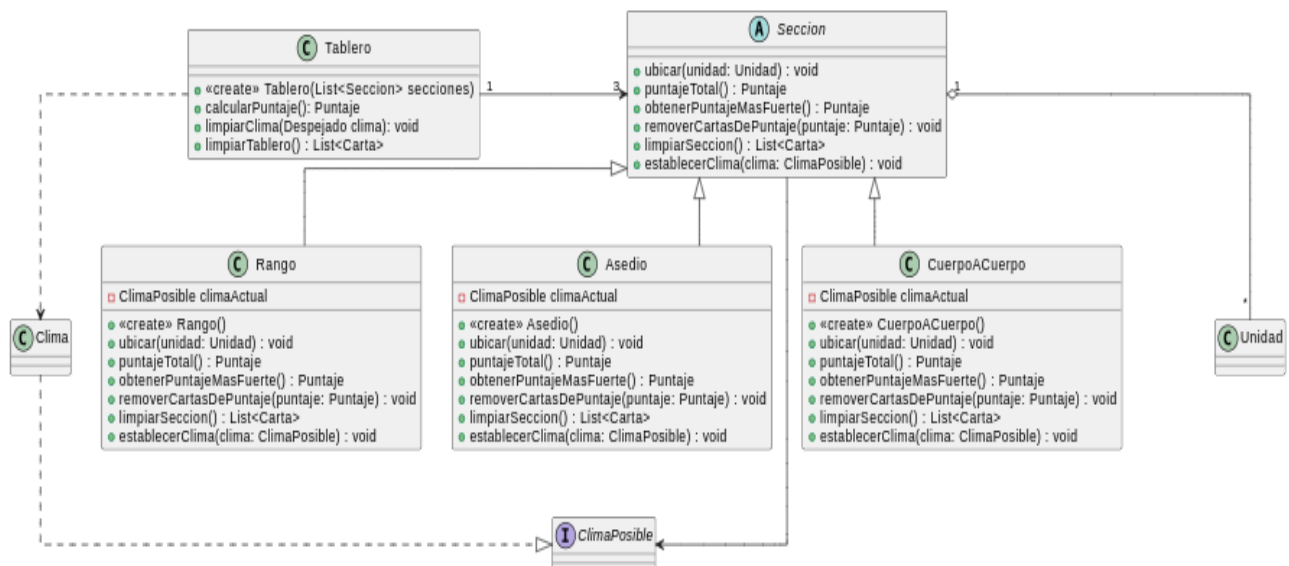


Figura 5: Diagrama con las secciones y tablero.

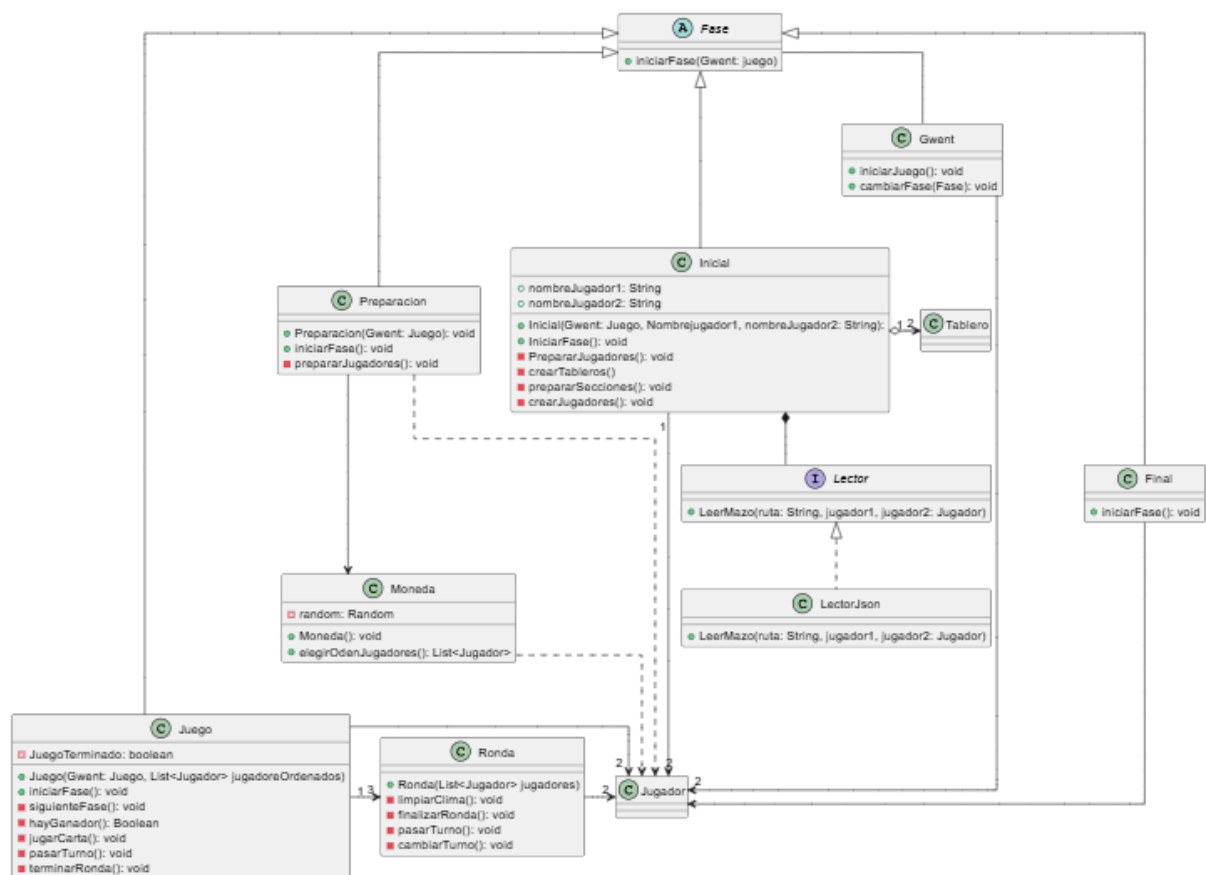


Figura 6: Diagrama de clase con el manejo del flujo del juego. Notemos que se está aplicando dos patrones: State en relación a las fases y luego Facade.

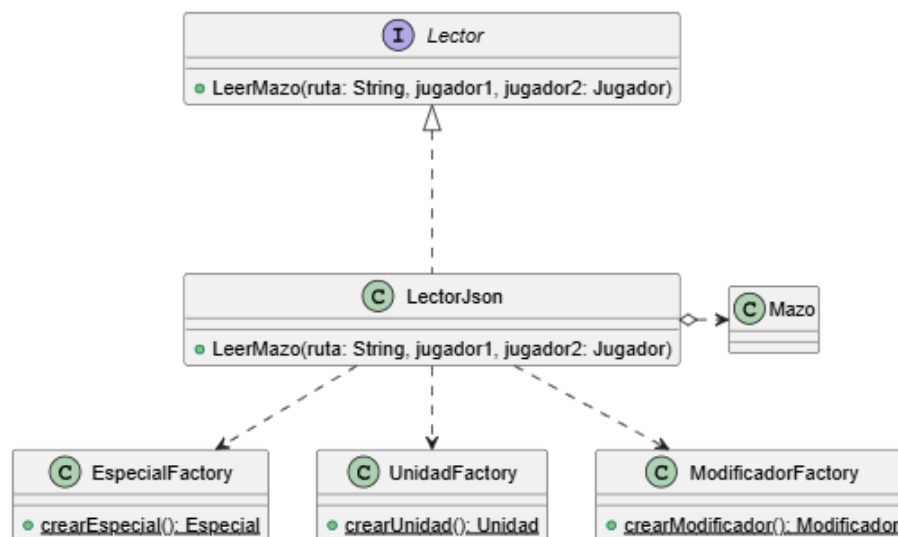


Figura 6: Diagrama de clase sobre la colaboración del lector con las fábricas. Se está aplicando Factory Method.

4. Detalles de implementación

El juego consta de 4 fases. Estas fases son las que interactúan con la clase Gwent que tendrá el manejo de todo el juego, y esta interactúa directamente con el Controlador. Para que en base a las acciones del usuario se pueda dar desarrollo a las Fases.

Luego la ronda se encargará de dar desarrollo a los turnos del jugador.

Utilizamos el patrón Null Object, para la parte de los modificadores. El objeto sinModificador utiliza dicho patrón.

También se hace uso del Null Object, para una cartaNula para usarse en correctamente en PilaDeDescarte.

5. Diagramas de secuencia

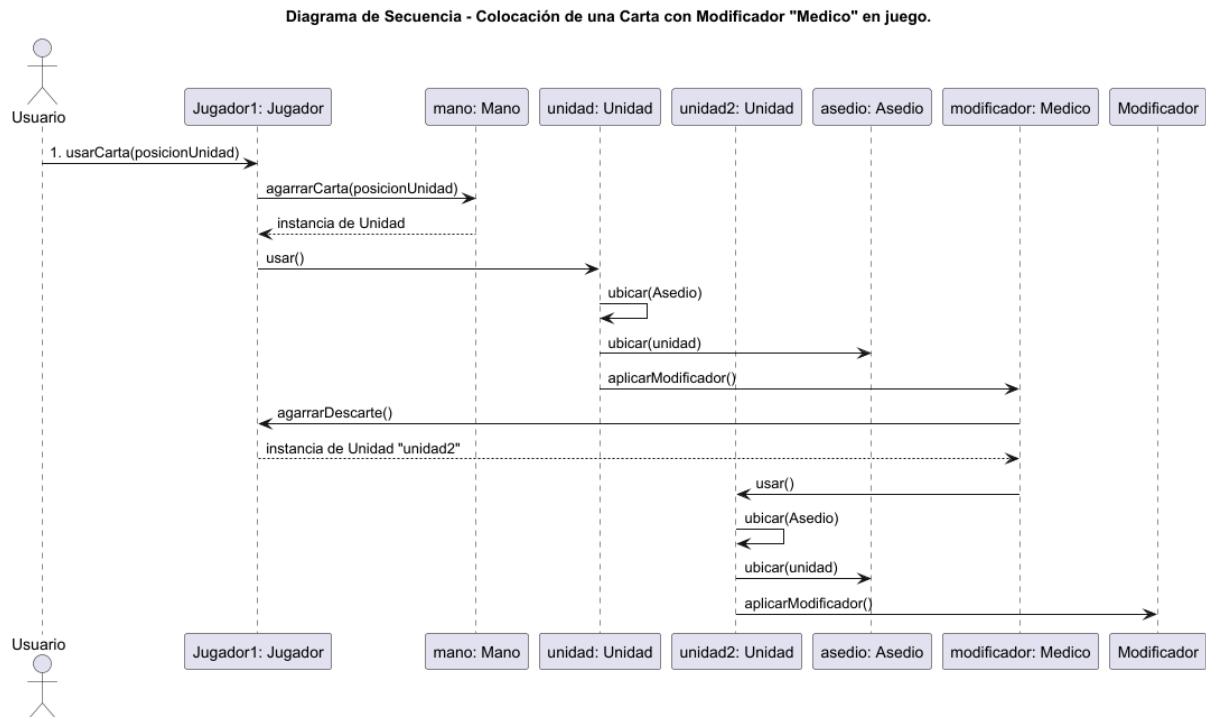


Figura 1: Un jugador elige usar una carta Unidad con modificador médico.

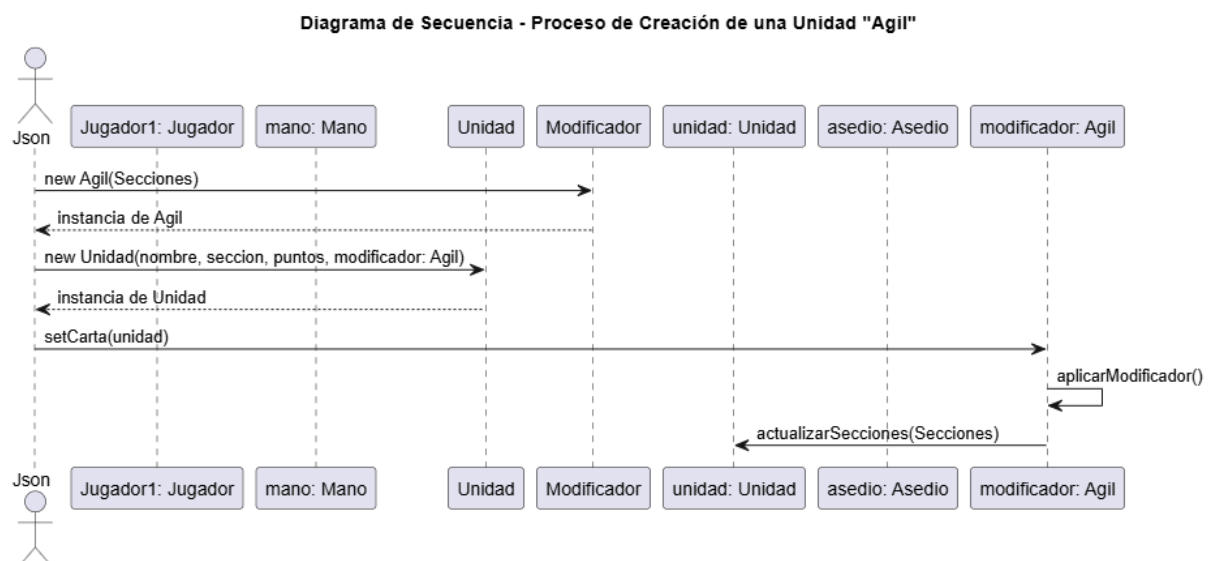


Figura 2: Proceso de creación de una Unidad con modificador Ágil.

Si bien el modificador ágil no aporta alguna mecánica interesante dentro del juego sino que es más representativo para indicar que una carta tiene dos o más secciones, para evitar que la clase Ágil quede anémica en nuestro modelo, se decidió que el modificador recibiera las secciones y que este mismo se encargue de modificarlas a la Unidad Asociada. Como hay una dependencia circular, necesariamente se introduce un método `setCarta()`, para que el modificador ágil la encapsule. Esto se aplicaría a la hora de instanciar.

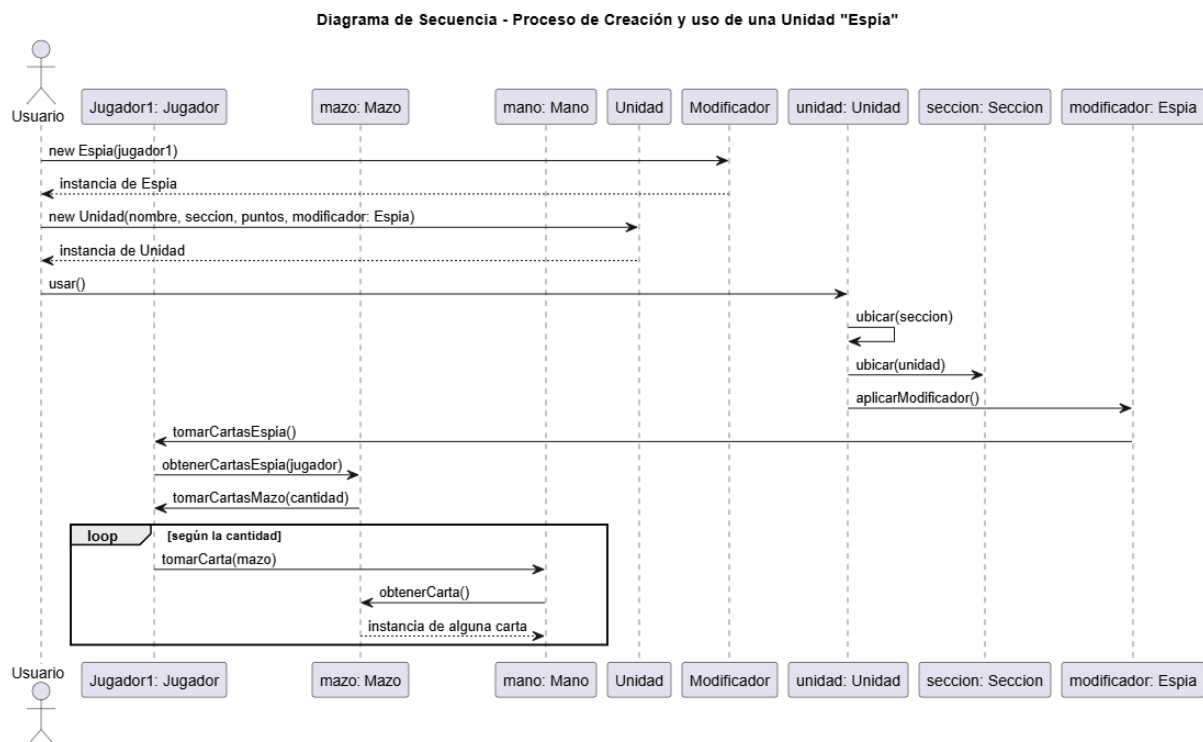


Figura 3: Proceso de creación y uso de una Unidad con modificador Espia.

Un detalle, es que la Unidad recibe la sección del jugador rival, ya que esta es la habilidad del espía: agarrar dos cartas del mazo al azar, pero con el costo de darle puntos al rival. Un balance interesante para esta carta sería que valga más puntos, así el costo de agarrar dos cartas aumenta notablemente.

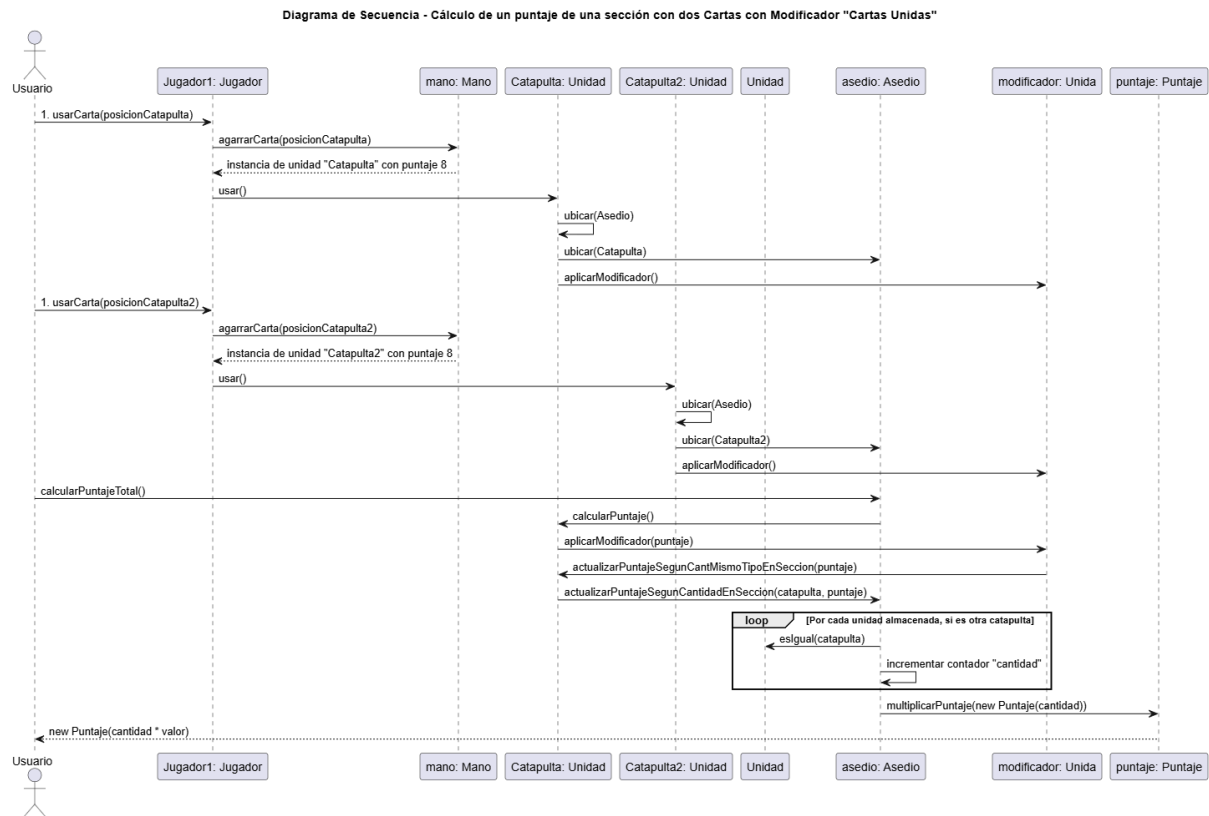


Figura 4: Cálculo de un puntaje de una sección con dos Cartas con Modificador Unida.

6. Diagrama de paquetes

Diagrama de Paquetes General

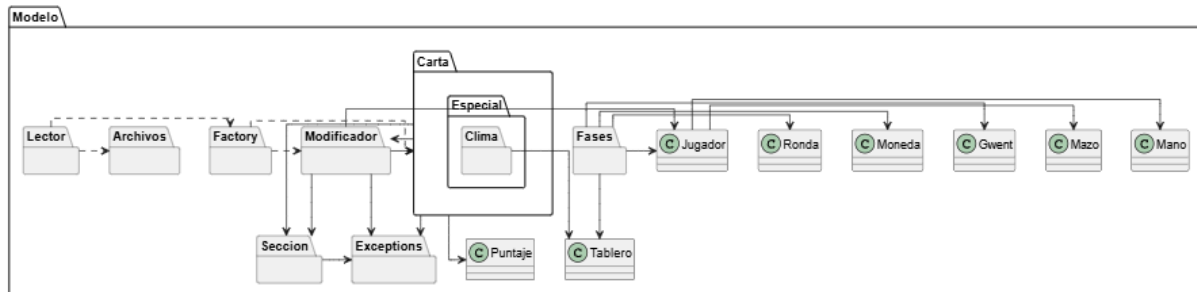


Figura 1: Diagrama del paquete Modelo

Diagrama de Paquetes General

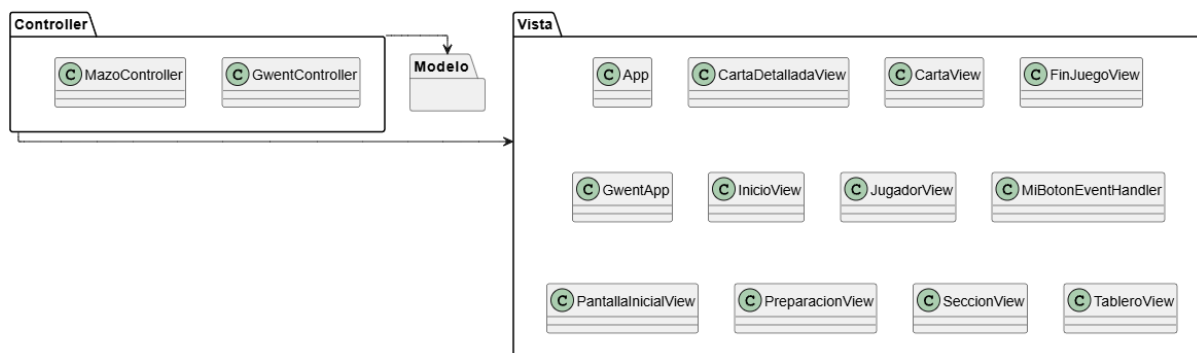


Figura 2: Diagrama de los paquetes, incluyendo vista y controladores.

Diagrama de Paquetes de Unidad - Modificador

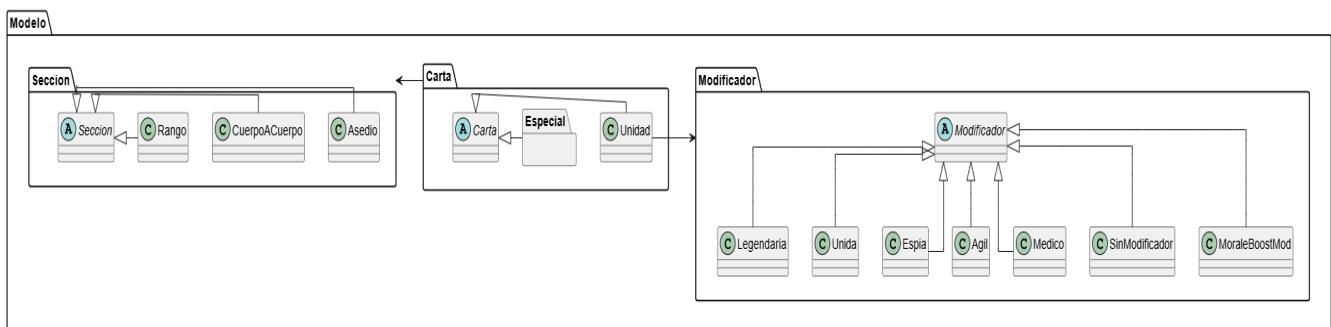


Figura 3: Diagrama de paquetes de la relación Unidad - Modificador.

7. Excepciones

noHayCartasSuficientesException: Se utiliza en la clase Mano para informar si cuenta con la cantidad de cartas necesarias. En caso de no ser así se lanza el mensaje "La mano no tiene cartas suficientes"

NoHayCartasUnidadSuficientesException: Se utiliza en la clase Mazo para verificar si la cantidad de cartas es la correcta. En caso de no ser así se lanza el mensaje "El mazo no tiene unidades suficientes".

NoHayCartasEspecialesSuficientesException: Se utiliza en la clase Mazo para verificar si la cantidad de cartas es la correcta. En caso de no ser así se lanza el mensaje "El mazo no tiene especiales suficientes".

UnidadNoSePuedeUbicarEnEstaSeccionException: Se utiliza en la clase Unidad para verificar que una unidad pueda ser ubicada en la sección que se la intenta usar. En caso de no ser así se lanza el mensaje "No se puede ubicar en esta sección".

SeccionInexistenteExcepcion: Se utiliza en la clase LectorJson para que en el caso de que la sección no exista en el Json arroje esta excepción.

ModificadorInexistenteException: Se utiliza en la clase ModificadorFactory para el caso en el que recibir un modificador que no existiese lance esta excepción.